

Lab 08: API Gateway, Redis Caching & VPC Security

1. Оршил (Introduction)

Өмнөх лаборатори дээр бид системээ Cloud-д деплой хийж, үйлчилгээ бүрийг өөрийн гэсэн Public IP хаягтайгаар ашигласан. Гэвч бодит системд бүх сервисээ интернэтэд ил тавих нь аюулгүй байдлын хувьд эрсдэлтэй юм.

Энэ лабораторийн ажлаар бид системээ илүү мэргэжлийн түвшинд шилжүүлж:

- API Gateway:** Бүх хүсэлтийг нэг цэгээр хүлээн авч, дотоод сервисүүд рүү "Proxy" хийнэ.
- Redis Cache:** Gateway дээр өгөгдлийг түр хадгалж (caching), системийн хурдыг нэмэгдүүлнэ.
- VPC (Virtual Private Cloud):** Дотоод сервисүүдийг интернэтээс тусгаарлаж, зөвхөн Gateway-ээр дамжуулан ханддаг болгоно.

2. Архитектурын бүтэц (Network Architecture)

```
graph TD
  Client[Frontend / Client] -->|Public Internet| GW[API Gateway]
  subgraph VPC [DigitalOcean VPC]
    GW -->|Private Network| JSON[User JSON Service]
    GW -->|Private Network| SOAP[User SOAP Service]
    GW -->|Private Network| FM[File Manager Service]
    GW <-->|Localhost| Redis[(Redis Cache)]
  end
  JSON --- DB[(Cloud DB)]
```

3. Даалгавар 1: Local Redis Setup

Gateway суулгасан Droplet дээрээ Redis-ийг суулгаж тохируулна.

- Суулгах (Ubuntu/Debian):** `bash sudo apt update sudo apt install redis-server`
- Тохиргоо шалгах:** `bash redis-cli ping` # Хариу: PONG
- Gateway-тэй холбох:** Таны Gateway програм `localhost:6379` хаягаар Redis-тэй холбогдож ажиллах ёстой.

4. Даалгавар 2: API Gateway хөгжүүлэлт (Polyglot)

Та өөрийн сонгосон хэл (Node.js, Java, Python, г.м)-ээр Gateway-г хөгжүүлнэ.

4.1. Үндсэн логик (Core Logic)

Routing: Ирж буй хүсэлтийн хаягаар нь ялгаж тохирох сервис рүү дамжуулна.

- `/api/users/**` -> `http://JSON_SERVICE_PRIVATE_IP:8080`
- `/api/soap/**` -> `http://SOAP_SERVICE_PRIVATE_IP:8080`

Caching (Redis): GET хүсэлтүүдийг Redis-д хадгална.

- Хүсэлт ирэхэд: Эхлээд Redis-ээс хайна (**Cache Hit**).
- Redis-д байхгүй бол: Үндсэн сервис рүү хандаж өгөгдлийг аваад Redis-д хадгална (**Cache Miss**).

4.2. Pseudocode Жишээ:

```
// Middleware logic
if (req.method === "GET") {
  let cachedResponse = await redis.get(req.url);
  if (cachedResponse) {
    return res.send(JSON.parse(cachedResponse));
  }
}

// After fetching from backend:
await redis.setEx(req.url, 60, JSON.stringify(backendData)); // 60 sec TTL
```

5. Даалгавар 3: VPC ба Аюулгүй байдал (VPC & Firewalls)

5.1. VPC тохируулах

1. DigitalOcean Dashboard -> **Networking** -> **VPC**.
2. Сонгосон Droplet-үүдээ нэг VPC-д (жишээ нь `sgp1-default`) оруулсан эсэхээ шалгаарай.

5.2. Cloud Firewalls

Дотоод сервисүүдийг хамгаалах:

1. Шинэ Firewall үүсгэнэ (Жишээ нь: [Internal-Service-Firewall](#)).

Inbound Rules:

- Port [8080](#), [80](#) гэх мэт сервисүүдийн портыг нээх.
- **Sources:** Зөвхөн Gateway-ийн **Private IP** эсвэл Gateway-ийн нэрийг (Droplet tag) сонгож өгнө.

3. Энэ Firewall-ийг JSON, SOAP, File Manager Droplet-үүдэд онооно.

6. Даалгавар 4: Frontend-ийг шинэчлэх

Frontend програм нь одоо сервиүүдийн тус тусын IP-рүү бус, харин цорын ганц **Gateway URL**-руу бүх хүсэлтээ явуулах ёстой.

- Хуучин: <http://157.x.x.x:8080/users>
- Шинэ: http://GATEWAY_IP/api/users

7. БОНУС: SOAP Cache Management

SOAP сервис нь Stateful болон Stateless холимог байх тохиолдол их байдаг. SOAP хүсэлт/хариуг Redis-д хэрхэн үр дүнтэй хадгалах болон "Cache Invalidation" (өөрчлөлт орох үед кэшийг устгах) стратегийг бодож олж, хэрэгжүүлбэл бонус оноо өгнө.

8. Илгээх зүйлс (Deliverables)

PDF тайланд:

1. **Architecture Screenshot:** DigitalOcean дээрх VPC дотор орсон Droplet-үүдийн жагсаалт.
2. **Firewall Screenshot:** Зөвхөн Gateway-ээр хязгаарласан Inbound Rules.
3. **Logs:** Gateway дээр "Cache Hit" болон "Cache Miss" болж буйг харуулсан Log зураг.
4. **Performance Check:** Өгөгдлийг кэшээс авч буй үед хүсэлтийн хугацаа багассаныг (Postman Time) харуулсан харьцуулалт.

Хугацаа: 1 долоо хоног. **Илгээх формат:** Lab08_SOA_[Таны_Код].pdf